Biçimsel Diller ve Otomatlar

Sets, Relations kümeler, bağıntılar

* Bir küme nesnelerden oluşur.
* Elemanların sırası ve tekrarı önemli değildir. — {r,b,c} ile {r,b} aynı.
* bir elemana sahipse Singleton, hiç elemanı yoksa empty olarak adlandırılır.
* Sonsuz küme
* küme özellikleriyle de tanımlanabilir. — G= {x: x∈ N and x is less than 10}.
* Alt küme — A⊆B ⇒ A=B ✓ — A⊂B ⇒ A≠B. proper.

Union = Birleşim ∪

Intersection = Kesişim ∩

Difference = Fark −

Disjoint = Ayrık küme A∩B = { }, ∅

DeMorgan's laws

A-(B∪C) = (A-B) ∩ (A-C)

A-(B∩C) = (A-B) ∪ (A-C)

Power set = Bir kümenin boş küme dahil tüm alt kümeleri 2^A, A'nın pas'i

Partition = Π power kümesinin alt kümesidir. boş kümeyi içermez. ve A kümesinin her elemanını bir kez bulundurur. Her eleman ayrıktır. — {{a,b}, {c,d}}

Ordered Pair sıralı ikili

Nesneler arasındaki ilişki kümelerle gösterilmez. sıralı çiftler (ordered pair) ile gösterilir. — (a,b)

Kartezyen Çarpımı

A x B ile gösterilir ve sonuç sıralı ikili ile gösterilir.

{1,3} x {b,c} = {(1,b), (1,c) (3,b), (3,c)}.

Discrete Mathematics

Binary relation ikili bağıntı

A x B'nin alt kümesidir

Tuples and relations

$(a_1, a_2, a_3 - a_n)$ ordered tuple olarak adlandırılır (n-tuple) (n'li) (n-ary)

Function = Fonksiyon $f: A \to B$

Domain ve Image = A domain, $f(a)$ image olarak adlandırılır ve her a unique'tir.

Arguments ve Value = $f(a_1, a_2 \dots a_n) = b$ için a_i arguments ve b value olarak adlandırılır

One to One = Birebir fonksiyon. $f(a) \neq f(a')$

Onto = Örten fonksiyon.

Bijection = Hem örten hem birebir fonksiyon.

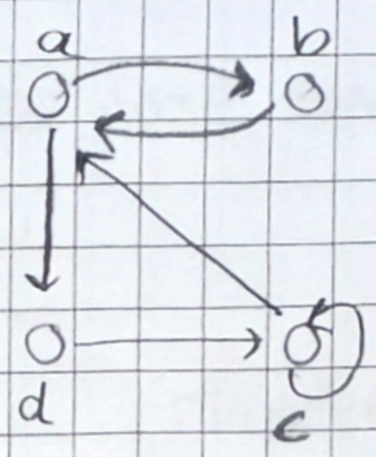
Inverse = ters fonksiyon

Graph

$R \subseteq A \times A$ ise A üzerinde bir binary ilişki için directed graph kullanılır

Graph üzerinde her bir node A'nın bir elemanıdır.

Her $(a, b) \in R$ için a'dan b'ye bir ok (kenar - edge) çizilir.



$R = \{(a,b), (b,a), (a,d), (d,c), (c,c), (c,a)\}$ ilişkisine ait graph.

Reflexive Yansımalı

$R \subseteq A \times A$, $a \in A$ $\forall a$ için $(a,a) \in R \Rightarrow$ reflexive

Symmetric Simetrik

$R \subseteq A \times A$, $a,b \in A$ ve $\forall (a,b)$ için $(b,a) \in R \Rightarrow$ symmetric

Antisymmetric Antisimetrik

$R \subseteq A \times A$, $a,b \in A$ ve $\forall (a,b)$ için $(b,a) \notin R \Rightarrow$ antisymmetric

Transitive Geçişken

$R \subseteq A \times A$, $(a,b) \in R$ ve $(b,c) \in R$ iken $(a,c) \in R \Rightarrow$ transitive

Equivalence relation Denklik İlişkisi

Bir ilişki reflexive, symmetric ve transitive ise equivalence rel. denir.

Partial order Kısmi Sıralama

Bir ilişki reflexive, antisymmetric ve transitive ise partial order denir.

Total order Tam sıralama

Bir partial order $\subseteq$ total order ilişkisi vardır. Kısmiden farklı olarak tam sıralamada A kümesinin tüm elemanları birbiriyle ilişkilenmelidir.

Path Yol

Bir binary ilişkideki path $(a_1, a_2 - a_n)$ sıralı serisidir ve bu seride her $(a_i, a_{i+1}) \in R$'dir

Length

Bir yol $(a_1, a_2 - a_n)$ için length n'dir.

Cycle

Bir yol $(a_1, a_2 \cdots a_n)$ ve $(a_n, a_1) \in R$ ise cycle'dır.

Reflexive transitive closure

Bir R ilişkisi reflexive ve transitive değilken, R'yi içeren R^* transitive ise R^* ilişkisi R ilişkisinin reflexive transitive closure'u denir. (R^* mümkün olan en az kümeden sağlıyor.)

ALPHABETS and LANGUAGES

* Bir alfabe sonlu sayıda sembolden oluşur. Roman = {a,b...z}, binary = {0,1}
* String bir alfabede tanımlanan sonlu sayıda sembolün sıralanışına denir.
* Sembol içermeyen string empty string olarak adlandırılır e ile gösterilir.
* Σ alfabe için Σ^* ise o alfabede boş string dahil tüm stringlerin kümesini gösterir.
* bir stringin uzunluğu(length) değeri |w| şeklinde gösterilir.
* concatenation iki stringin ardarda eklenmesidir. w = xay ile gösterilir.

Substring = v stringi w içerisinde w = xvy şeklinde yer alır. x,y = e olabilir.

Suffix = v stringi w içerisinde w = xv şeklinde yer alır.

Prefix = v stringi w içerisinde w = vy şeklinde yer alır.

Örnek:

v = road için;

roadrunner → Prefix

abroad → Suffix

broader → Substring, şeklinde tanımlanır.

* w^i bir stringin ne kadar tekrar ettiğini gösterir. $ab^2 = abab$
* w^R bir stringin tersini (reverse) ifade eder. w = aay → w^R = yaa
* Bir alfabe Σ üzerinde tanımlı string kümesi Σ^* kümesinin alt kümesidir ve Language (dil) denir.
* Σ, Σ^* ve $\emptyset$ birer dildir. - {aba, ctr, d, f} bir sonlu dildir ve Σ = {a,b...z} alfabesi üzerinde tanımlıdır.
* L^* bir dildeki tüm stringleri, L^+ ise boş str hariç tüm strleri ifade eder.
* regular expression (düzenli ifade)'lar bir dili doğrudan $\emptyset$, U ve * ile tanımlar.

$L = 0^*10^*01^*(10^* \cup \emptyset)$

* a bir regular ex için $\mathcal{L}(a)$, a tarafından tanımlanan dili ifade eder.

* a bir regex için $L = \mathcal{L}(a)$ şeklinde tanımlanabilen dillere regular languages (düzenli dil) denir.
* bir w stringin bir L diline ait olup olmadığını bulan algoritmaya language recognition device diyi tanıyan cihaz denir.

(DFA) Deterministic Finite Automata deterministik sonlu otomatlar

* bir finite automaton(sonlu otomat) kabit ve sonlu kapasiteye sahip bir mekanik işlem-
* giriş bilgisini string olarak bir input tape üzerinden alır.
* bir çıkış üretmez, giriş bilgisinin kabul edilip edilmediğini gösterir
* compiler'da lexical analysis (begin, +, for) aşamasında kullanılır. dil tanıyıcı cihazdır.
* Otomat her seferinde input tapeden bir sembol okur ve yeni duruma geçer.
* yeni duruma geçme sadece mevcut durum ile okunan değere bağlıdır.
* string bittiğinde final state(s), sonuç durumlarından birindeyse kabul edilir.

DFA tanım:

Bir DFA quintuple olarak tanımlanır $M = (K, \Sigma, \delta, s, F)$

K = sonlu sayıda durumlar kümesi,

Σ = alfabe

δ = transition function (geçiş fonk.) $K \times \Sigma$ dan K'ya

s = $s \in K$ başlangıç durumu (sadece bir tane)

F = $F \subseteq K$ final state(s) kümesi

δ (mevcut d., okunan değ.) şeklinde işlev yapar.

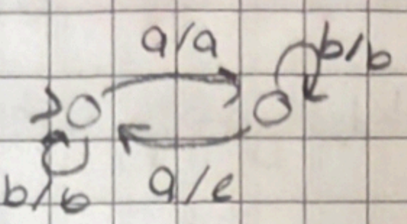
Configuration = otomatın herhangi bir durumu ile input tape'nin sağ kısmındaki str'yi ifade eder

* conf. $K \times \Sigma^*$'ın bir elemanıdır → $(q_2, ababab)$
* $\vdash_M$ ardarda iki konfigürasyon arasında binary relation'ı ifade eder.
* $(q, w) \vdash_M (q', w')$ için $w = aw'$, $a \in \Sigma \Rightarrow \delta(q, a) = q'$ olmak zorundadır.

* t_M fonksiyonu $K \times E^+$'dan $K \times E^*$'a bir fonksiyondur.
* t_M fonksiyonunun reflexive, transitive closure'u $\vdash_M^*$ şeklinde tanımlanır.

Deterministic finite-state transducer

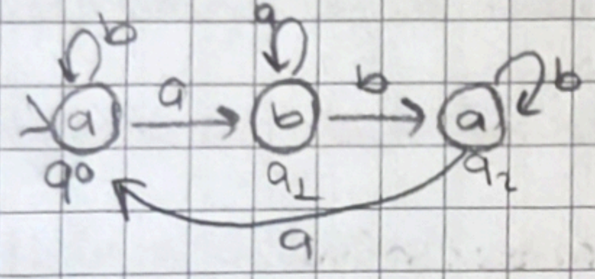
* giriş stringine göre bir çıkış stringi üreten sonlu otomattır.
* a/w girişten a okur ve çıkışa w yazar.

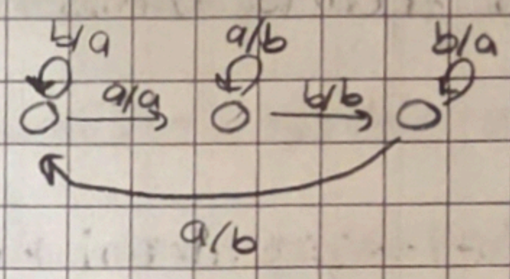


abbaab → abbeab

! bu otomatta F kümesi yoktur çünkü çıkış verir.

* bu otomat 2'ye ayrılır. moore ve mealy

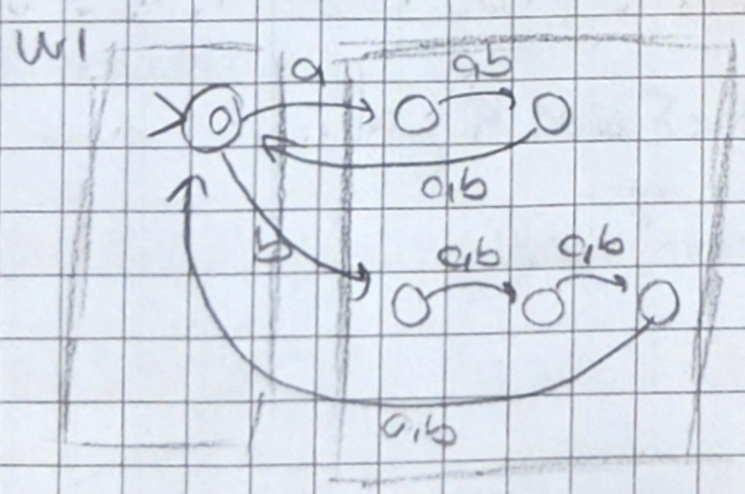




Deterministic 2-tape finite automata

String çiftlerini kabul eden ve iki girişe sahip olan otomattır.

Transition func. 1.tape ve 2.tape için farklıdır.



(aa, abba) ✓

(aba, aabaaab) ✓

(ab, abab) ✗

! stringler birbirine referans göstererek okunur. Sırayla.

(NFA) Nondeterministic Finite Automata

* NFA durum geçişlerinde sadece mevcut durum ve giriş bilgilerini kullanmaz
* aynı giriş bilgisi ve aynı durum için birden fazla sonraki durum olabilir
* bilgisayarların gerçek modellemesinde kullanılmazlar. Automata tanımlamasını basitleştirmek için
* Her nondeterministic otomatın deterministik karşılığı vardır.

NFA tanım:

bir NFA quintuple olarak tanımlanır. $M = (K, \Sigma, \Delta, s, F)$

K sonlu sayıda durumlar kümesi

Σ alfabe

Δ transition relation (fonk. değil) $K \times (\Sigma \cup \{e\}$ dan K'ya

s $\in K$ başlangıç durumu (sadece bir tane)

F $\subseteq K$ final state(s) kümesi

* Her $(q, u, p) \in \Delta$ üçlüsü M'nin geçişi olarak adlandırılır.
* M'nin configuration'ı $K \times \Sigma^*$ dır. (q, w) t_M (q', w') geçişi için $w = uw'$, $u \in \Sigma \cup \{\epsilon\}$ ve $(q, u, q') \in \Delta$ olmak zorundadır.
* t_M bir fonk. değildir çünkü bir (q, w) konf. için çok sayıda (q', w') olabilir.

Nondeterministic/Deterministic Finite Automata

* Her NFA'nın bir DFA'sı vardır.
* $M=(K,\Sigma,\Delta,s,F)$ bir NFA ve $M'=(K',\Sigma,\delta',s',F')$ DFA eşiti olsun.
* M toplam $\{q_0,q_1,q_2,q_3,q_4\}$ olmak üzere 5 duruma sahip olsun.
* M herhangi bir durumdayken girilen bir str için $\{q_0,q_2,q_3\}$ gibi çoklu durumda olabiliyorsa M' için tek bir durum olarak $q_x=\{q_0,q_2,q_3\}$ alınır.
* NFA'da q durumlarında bazılarına e-transition ile geçilebilir.
* M ve M' nün eşit olabilmesi için;

$w\in\Sigma^*$ ve $(s,w)\vdash_M^* (f,e)$, $f\in F$ için $(E(s),w)\vdash_{M'}^* (Q,e)$, Q'nun en az bir elemanı $f\in F$ olmak zorundadır. (M'nin kabul ettiği str'yi M'de kabul etmeli).

* $E(s)$ = s durumundan e-transition ile gidilebilecek her durumu temsil eder.
* Q = durumlar kümesi (örn) (q_0,q_3,q_4).
* düşünülen DFA M' için $K'=2^K$ kadardır. Ancak tüm durumlar kullanılmayabilir.
* q_x durumundan b ile geçilebilecek durum kümesi için, $\{q_0,q_2,q_3\}$ durumlarının a ile gittiği durumlar bulunur. $(q_0,b,q_2)(q_2,b,q_4)$ için (q_x,b,q_y)

$q_y=E(q_2)\cup E(q_4)$ olur

FINITE AUTOMATA and REGULAR EXPRESSIONS

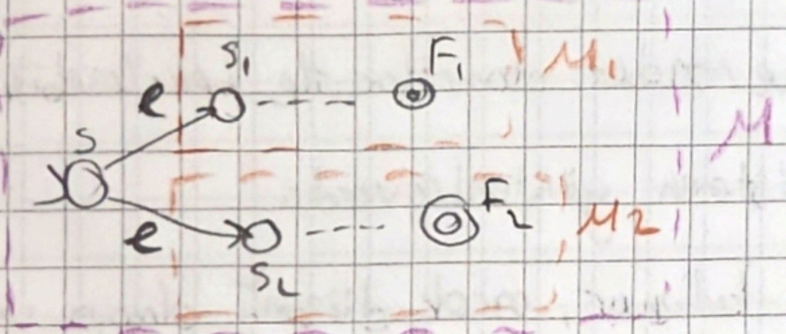
* Finite automata tarafından kabul edilen diller sınıfı;
  - Union $\cup$
  - Concatenation o
  - Kleene Star $*$
  - Complementation $\Sigma^* -$
  - Intersection $\cap$

Union

$M_1 = (K_1, \Sigma, \Delta_1, s_1, F_1)$ ve $M_2 = (K_2, \Sigma, \Delta_2, s_2, F_2)$ nondet. otomat olsun.

$L(M) = L(M_1) \cup L(M_2)$ olacak şekilde yeni bir automata M tanımlanabilir.

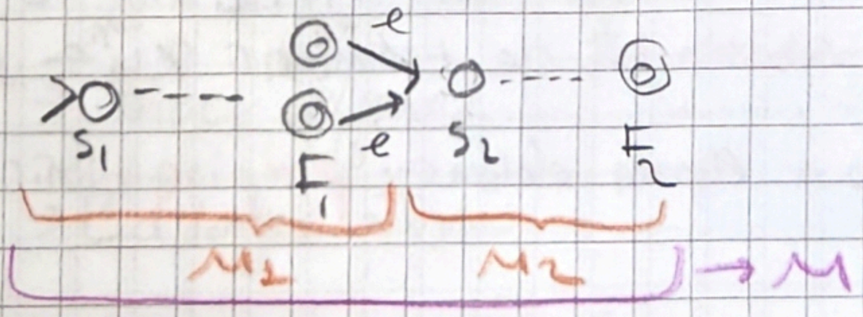
Not: M, M_1 ve M_2 arasında e-transition ile başlangıçta geçiş yapar.



Concatenation

M_1 M_2 nondet olmak üzere $L(M) = L(M_1) \circ L(M_2)$ şeklinde M tanımlanır.

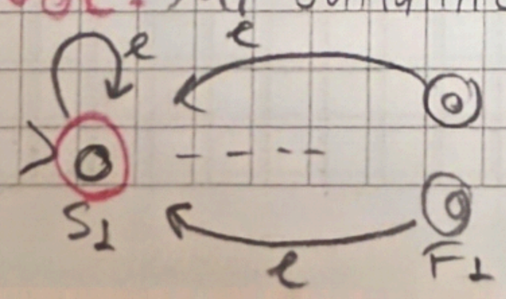
Not: M_1 sonlanınca e-transition ile M_2'ye geçiş yapar



Kleene Star

M_1 nondet olmak üzere $L(M) = L(M_1)^*$ şeklinde M tanımlanabilir.

Not: M_1 sonlanınca e-transition ile tekrar başlangıca geçebilir.



Complementation

$M = (K, \Sigma, \delta, s, F)$ deterministic otomat olsun

$\overline{L} = \Sigma^* - L(M)$ olacak şekilde yeni bir otomat tanımlanabilir

$\overline{M} = (K, \Sigma, \delta, s, K-F)$ olacak şekilde yeni bir M tanımlanabilir.

İntersection

$$L_1 \cap L_2 = \Sigma^* - ((\Sigma^* - L_1) \cup (\Sigma^* - L_2))$$

REGULAR and NON-REGULAR LANGUAGES

* Düzenli diller bazı işlemler (birleşim, kesişim, kleene star, ...) için kapalıdır.
* Düzenli diller regular exp., det ve nondet otomatlar ile belirlenebilir.
* Bir dilin düzenli olduğunu gösteren yöntemler vardır.
* İki özellik tüm düzenli dillerde bulunur, ancak düzenli olmaya dillerde de bulunmaz.
  - Otomatta bir str için gereken toplam hafızanın sınırı vardır. Bu sınır sabittir ve ilgili stringe değil dile bağlıdır.
  - Sonsuz sayıda stringe sahip olan düzenli diller, dönüşe sahip otomatlar veya kleene star içeren regular exp.lar tarafından gösterilebilir.
* En az bir noktada yol daha önce geçtiği durumlara, döner ve tekrar gecerse **loop** olur.
* Sonsuz sayıda str'ler xy^nz substringleri ile tanımlanır. $xy^nz, (n = 1, 2, 3 \dots)$
* bu stringe y^n kısmındaki loop a eleman eklemeye **pumping** denir.

Context-Free Grammers and Languages

Bir CFG, $G=(V,\Sigma,R,S)$ şeklinde tanımlanır.

$V \rightarrow$ alfabe (terminal ve nonterminaller)

$\Sigma \rightarrow$ terminaller $(\Sigma \subseteq V)$

$R \rightarrow$ kurallar $(V-\Sigma) \times V^*$

$S \rightarrow$ başlangıç sembolü $S \in (V-\Sigma)$

Bir CFL $L(G) = \{w \in \Sigma^* : S \Rightarrow_G^* w\}$ şeklindedir

$L = \{w | w = 0^* 1^*\} \Rightarrow$

$S \rightarrow AB$

$A \rightarrow 0A \mid \varepsilon$

$B \rightarrow 1B \mid \varepsilon$

Pushdown Automata

pushdown automata $M=(K,\Sigma,\Gamma,\Delta,s,F)$ şeklinde bir (6-tuple) ile tanım...

$K \rightarrow$ durumlar

$\Sigma \rightarrow$ giriş stringi için alfabe

$\Gamma \rightarrow$ stack alfabesi

$S \rightarrow$ Başlangıç durumu

$F \rightarrow$ sonuç durum kümesi

$\Delta \rightarrow$ geçiş ilişkisi $(K \times (\Sigma \cup \{e\}) \times \Gamma^*) \times (K \times \Gamma^*)$

Turing Makineleri

* inputlar bant(tape) üzerinden alınır.
* input tape sonsuzdur.
* 2 tane alfabe vardır. input ve Tape. □(boşluk) karakteri tape'ye ait.
* Kabul ve ret durumu vardır. Bunlar Halting State olarak geçer. bu durumlar makineyi durdurur.
* Her adımda bantta okuma, yazma, sağa sola hareket yapılabilir.
* ε(e) transiton yoktur.
* Deterministik makinelerdir.